Continuous-Time Models

Neural ODEs and Implicit Neural Representations

1 Motivation: Why Continuous-Time Models?

A time series is usually observed as a finite sequence

$$x_1, x_2, \ldots, x_T, \tag{1}$$

but many real-world signals are better understood as samples of an underlying continuous-time process

$$x(t), \qquad t \in \mathbb{R}. \tag{2}$$

For example, ECG signals, audio signals, sensor recordings, and physical trajectories evolve continuously, even if the data are stored at discrete timestamps.

In practice, time series data also suffer from issues such as:

- missing observations, because sensors may fail;
- irregular sampling, because measurements are not always collected at equally spaced times;
- variable-length sequences, because different subjects or systems may be observed for different durations;
- extrapolation requirements, because forecasting asks for future values beyond the observed interval.

Classical sequence models such as CNNs, RNNs, and attention-based models often assume either a fixed grid or a discrete sequence representation. Missing-data imputation can help, but it is an additional preprocessing step. Continuous-time models try to avoid this limitation by building neural architectures that can be queried directly at arbitrary timestamps.

The two main families studied in this chapter are:

1. **Neural Ordinary Differential Equations (Neural ODEs)**, which learn a continuous-time dynamical system;
2. **Implicit Neural Representations (INRs)**, which represent a signal as a neural function of time.

2 Ordinary Differential Equations

2.1 Definition

An ordinary differential equation describes how a state evolves continuously in time. The classical first-order formulation is

$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f(x(t), t), \tag{3}$$

with initial condition

$$x(0) = x_0. \tag{4}$$

Here:

- $x(t)$ is the state of the system at time t;
- $f(x(t), t)$ is the instantaneous velocity or rate of change;
- x_0 is the initial state.

The ODE answers the question:

$$\text{Given where I am now, how fast and in which direction do I move?} \tag{5}$$

Integrating both sides from 0 to t gives

$$\int_0^t \frac{\mathrm{d}x(s)}{\mathrm{d}s}\,\mathrm{d}s = \int_0^t f(x(s), s)\,\mathrm{d}s, \tag{6}$$

$$x(t) - x(0) = \int_0^t f(x(s), s)\,\mathrm{d}s. \tag{7}$$

Therefore,

$$\boxed{x(t) = x_0 + \int_0^t f(x(s), s)\,\mathrm{d}s.} \tag{8}$$

This integral form is the basis of numerical ODE solvers.

2.2 Example 1: Exponential Growth or Decay

Consider the scalar ODE

$$\dot{x}(t) = ax(t), \qquad x(0) = x_0. \tag{9}$$

Its solution is

$$\boxed{x(t) = x_0 e^{at}.} \tag{10}$$

Indeed,

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(x_0 e^{at}\right) = a x_0 e^{at} = ax(t). \tag{11}$$

If $a > 0$, the system grows exponentially. If $a < 0$, the system decays exponentially.

2.3 Example 2: State- and Time-Dependent Dynamics

A simple example where the derivative depends on both $x(t)$ and t is

$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = -x(t) + \sin(t), \qquad x(0) = 0. \tag{12}$$

Here

$$f(x, t) = -x + \sin(t). \tag{13}$$

The term $-x(t)$ pulls the state toward zero, while $\sin(t)$ acts as a time-dependent periodic forcing.

This ODE is linear non-homogeneous:

$$x'(t) + x(t) = \sin(t). \tag{14}$$

Multiplying by the integrating factor e^t gives

$$e^t x'(t) + e^t x(t) = e^t \sin(t). \tag{15}$$

The left-hand side is

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(e^t x(t)\right). \tag{16}$$

Thus

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(e^t x(t)\right) = e^t \sin(t). \tag{17}$$

Integrating,

$$e^t x(t) = \int e^t \sin(t)\,\mathrm{d}t + C. \tag{18}$$

Using

$$\int e^t \sin(t)\,\mathrm{d}t = \frac{e^t}{2}\left(\sin(t) - \cos(t)\right), \tag{19}$$

we get

$$x(t) = \frac{1}{2}\sin(t) - \frac{1}{2}\cos(t) + Ce^{-t}. \tag{20}$$

The initial condition $x(0) = 0$ gives

$$0 = -\frac{1}{2} + C, \qquad C = \frac{1}{2}. \tag{21}$$

Therefore,

$$\boxed{x(t) = \frac{1}{2}\sin(t) - \frac{1}{2}\cos(t) + \frac{1}{2}e^{-t}.} \tag{22}$$

3 ODE Solvers

3.1 Why Numerical Solvers Are Needed

The exact solution of

$$x(t) = x_0 + \int_0^t f(x(s), s)\,\mathrm{d}s \tag{23}$$

may not be computable analytically. This is especially true when f is a neural network. An ODE solver approximates the trajectory at discrete points

$$t_0, t_1, \ldots, t_N, \tag{24}$$

with step size

$$h = t_{n+1} - t_n. \tag{25}$$

We denote

$$x_n \approx x(t_n). \tag{26}$$

3.2 Euler Method

The Taylor expansion gives

$$x(t+h) = x(t) + h\dot{x}(t) + \frac{h^2}{2}x''(t) + O(h^3). \tag{27}$$

Keeping only the first-order term and using $\dot{x}(t) = f(x(t), t)$ gives

$$x(t+h) \approx x(t) + hf(x(t), t). \tag{28}$$

Thus the Euler method is

$$\boxed{x_{n+1} = x_n + hf(x_n, t_n).} \tag{29}$$

3.2.1 Euler Example

Consider

$$\dot{x}(t) = -x(t), \qquad x(0) = 1. \tag{30}$$

Euler gives

$$x_{n+1} = x_n + h(-x_n) = (1-h)x_n. \tag{31}$$

For $h = 0.1$,

$$x_{n+1} = 0.9x_n. \tag{32}$$

Starting from $x_0 = 1$,

$$x_1 = 0.9, \qquad x_2 = 0.81, \qquad x_3 = 0.729. \tag{33}$$

The exact values are

$$e^{-0.1} \approx 0.9048, \qquad e^{-0.2} \approx 0.8187, \qquad e^{-0.3} \approx 0.7408. \tag{34}$$

Euler is simple, but its accuracy is limited.

3.3 Runge–Kutta Methods

Euler uses only one slope, $f(x_n, t_n)$, at the beginning of the interval. Runge–Kutta methods estimate the average slope more carefully by evaluating f multiple times inside the interval $[t_n, t_{n+1}]$.

3.3.1 RK2: Midpoint Method

The midpoint method computes

$$k_1 = f(x_n, t_n), \tag{35}$$

$$k_2 = f\left(x_n + \frac{h}{2}k_1,\, t_n + \frac{h}{2}\right), \tag{36}$$

$$x_{n+1} = x_n + hk_2. \tag{37}$$

The interpretation is:

1. compute an initial slope k_1;
2. use k_1 to estimate the midpoint state;
3. compute the slope k_2 at that midpoint;
4. use the midpoint slope for the full step.

3.3.2 RK4: Classical Fourth-Order Method

The classical RK4 method computes four slopes:

$$k_1 = f(x_n, t_n), \tag{38}$$

$$k_2 = f\left(x_n + \frac{h}{2}k_1,\, t_n + \frac{h}{2}\right), \tag{39}$$

$$k_3 = f\left(x_n + \frac{h}{2}k_2,\, t_n + \frac{h}{2}\right), \tag{40}$$

$$k_4 = f(x_n + hk_3,\, t_n + h). \tag{41}$$

Then

$$\boxed{x_{n+1} = x_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).} \tag{42}$$

The weighted average gives more importance to the two midpoint slopes. RK4 is often much more accurate than Euler for the same step size, but it requires four evaluations of f per step.

4 Error Analysis for ODE Solvers

4.1 Meaning of $O(h^p)$

Writing

$$\text{error} = O(h^p) \tag{43}$$

means that there exists a constant $C > 0$ such that, for sufficiently small h,

$$|\text{error}| \leq Ch^p. \tag{44}$$

Thus, the error behaves like h^p as h becomes small.

If we halve the step size,

$$h \mapsto \frac{h}{2}, \tag{45}$$

then

$$h^p \mapsto \left(\frac{h}{2}\right)^p = \frac{h^p}{2^p}. \tag{46}$$

Therefore:

- if the global error is $O(h)$, halving h divides the error by about 2;
- if the global error is $O(h^2)$, halving h divides the error by about 4;
- if the global error is $O(h^4)$, halving h divides the error by about 16.

4.2 Local and Global Error

The *local truncation error* is the error made in one step, assuming the current value is exact. The *global error* is the accumulated error over the whole interval.

If a method has local error

$$O(h^{p+1}), \tag{47}$$

then over an interval $[0, T]$ the number of steps is approximately

$$N \approx \frac{T}{h}. \tag{48}$$

The accumulated global error is therefore roughly

$$\frac{1}{h} O(h^{p+1}) = O(h^p). \tag{49}$$

Thus a method of global order p has local error of order $p + 1$.

4.3 Euler Error

The exact Taylor expansion is

$$x(t+h) = x(t) + hf(x(t),t) + \frac{h^2}{2}x''(t) + O(h^3). \tag{50}$$

Euler uses only

$$x(t) + hf(x(t),t). \tag{51}$$

Thus it ignores terms of order h^2 and higher. Therefore,

$$\text{local error of Euler} = O(h^2), \tag{52}$$

and

$$\text{global error of Euler} = O(h). \tag{53}$$

4.4 RK2 Error

For the midpoint RK2 method,

$$k_2 = f\left(x(t) + \frac{h}{2}f(x(t),t), t + \frac{h}{2}\right). \tag{54}$$

This approximates the slope at the midpoint. Using Taylor expansion,

$$k_2 = x'(t) + \frac{h}{2}x''(t) + O(h^2). \tag{55}$$

Then

$$x_{n+1} = x(t) + hk_2 \tag{56}$$

$$= x(t) + hx'(t) + \frac{h^2}{2}x''(t) + O(h^3). \tag{57}$$

The exact solution is

$$x(t+h) = x(t) + hx'(t) + \frac{h^2}{2}x''(t) + \frac{h^3}{6}x'''(t) + O(h^4). \tag{58}$$

RK2 matches terms up to order h^2, so

$$\text{local error of RK2} = O(h^3), \tag{59}$$

and

$$\text{global error of RK2} = O(h^2). \tag{60}$$

4.5 Comparison

Method	Function evaluations per step	Local error	Global error
Euler	1	$O(h^2)$	$O(h)$
RK2	2	$O(h^3)$	$O(h^2)$
RK4	4	$O(h^5)$	$O(h^4)$

In Neural ODEs, each function evaluation is a neural network evaluation. Hence higher-order solvers may reduce numerical error, but they can increase computational cost. The cost is often measured by the number of function evaluations, denoted NFE.

5 Neural ODEs

5.1 Classical Formulation

A classical ODE assumes that the dynamics are known:

$$\frac{\mathrm{d}z(t)}{\mathrm{d}t} = f(z(t), t). \tag{61}$$

A Neural ODE replaces the unknown dynamics by a neural network:

$$\boxed{\frac{\mathrm{d}z(t)}{\mathrm{d}t} = f_\theta(z(t), t), \qquad z(t_0) = z_0.} \tag{62}$$

The neural network f_θ takes as input the current state and time,

$$(z(t), t), \tag{63}$$

and outputs the instantaneous velocity

$$f_\theta(z(t), t) \approx \frac{\mathrm{d}z(t)}{\mathrm{d}t}. \tag{64}$$

The ODE solver then integrates this velocity field:

$$z(t_1) = z(t_0) + \int_{t_0}^{t_1} f_\theta(z(t), t)\,\mathrm{d}t. \tag{65}$$

This is written compactly as

$$\boxed{z(t_1) = \mathrm{ODESolve}(z(t_0), f_\theta, t_0, t_1).} \tag{66}$$

5.2 What Is the Mapping?

A standard neural network learns a direct mapping

$$x \mapsto y. \tag{67}$$

A Neural ODE learns dynamics

$$(z, t) \mapsto f_\theta(z, t), \tag{68}$$

and the solver converts these dynamics into a map between states:

$$z(t_0) \mapsto z(t_1). \tag{69}$$

Thus the model mapping is

$$\boxed{z(t_0) \mapsto \mathrm{ODESolve}(z(t_0), f_\theta, t_0, t_1).} \tag{70}$$

5.3 Training a Neural ODE

Suppose we observe two states $z(t_0)$ and $z(t_1)$. The prediction is

$$\widehat{z}(t_1) = \mathrm{ODESolve}(z(t_0), f_\theta, t_0, t_1). \tag{71}$$

A simple squared-error loss is

$$\mathcal{L}(\theta) = \|\widehat{z}(t_1) - z(t_1)\|_2^2 . \tag{72}$$

For several observation times $t_1, \ldots, t_N$, we can write

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \|\text{ODESolve}(z(t_0), f_\theta, t_0, t_i) - z(t_i)\|_2^2 . \tag{73}$$

Training updates the parameters of f_θ:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta). \tag{74}$$

The key technical point is that $\mathcal{L}$ depends on θ through the ODE solver. Therefore, training requires computing gradients through the solver.

5.4 Connection with ResNets

Using Euler discretization,

$$z_{k+1} = z_k + h f_\theta(z_k, t_k). \tag{75}$$

A residual network layer has the form

$$z_{k+1} = z_k + F_{\theta_k}(z_k). \tag{76}$$

Thus Euler-discretized Neural ODEs resemble ResNets. The difference is that Neural ODEs use the same vector field f_θ across continuous time, while standard ResNets may use different parameters across layers.

5.5 Adjoint Sensitivity Idea

Define the adjoint state

$$a(t) = \frac{\partial \mathcal{L}}{\partial z(t)}. \tag{77}$$

It measures the sensitivity of the loss to the state at time t. Instead of storing all intermediate solver states and backpropagating through each numerical step, the adjoint method solves a backward-in-time sensitivity equation. Conceptually:

$$\text{forward pass:} \quad \text{solve for } z(t), \tag{78}$$

$$\text{backward pass:} \quad \text{solve for sensitivities } a(t). \tag{79}$$

This is important because Neural ODEs can involve many internal solver steps, especially with adaptive solvers.

6 Latent Neural ODEs

6.1 Why Latent NODEs?

A basic Neural ODE evolves directly in input space:

$$\frac{\mathrm{d}x(t)}{\mathrm{d}t} = f_\theta(x(t), t). \tag{80}$$

For time series forecasting, this can be limiting: reconstructing or forecasting from a single observation $x(t_0)$ may not contain enough information about the future. A single ECG value, for example, does not determine the next full wave.

Latent Neural ODEs solve this by:

1. encoding the observed history into a latent state;
2. evolving this latent state with an ODE;
3. decoding the latent trajectory back to the data space.

6.2 Mathematical Formulation

Suppose we observe

$$x(t_0), x(t_1), \ldots, x(t_N). \tag{81}$$

An encoder, often an RNN, summarizes the observations into a latent initial state:

$$q_\psi(z_0 \mid x(t_0), \ldots, x(t_N)). \tag{82}$$

A probabilistic latent NODE may output

$$q_\psi(z_0 \mid x_{0:N}) = \mathcal{N}(\mu_\psi, \Sigma_\psi). \tag{83}$$

Then

$$z_0 \sim q_\psi(z_0 \mid x_{0:N}). \tag{84}$$

The latent state evolves according to

$$\frac{\mathrm{d}z(t)}{\mathrm{d}t} = f_\theta(z(t), t). \tag{85}$$

Thus

$$z(t_i) = \mathrm{ODESolve}(z_0, f_\theta, t_0, t_i). \tag{86}$$

Finally, a decoder maps latent states back to observations:

$$\widehat{x}(t_i) = g_\phi(z(t_i)). \tag{87}$$

A simple deterministic loss is

$$\mathcal{L}(\psi, \theta, \phi) = \sum_i \left\| x(t_i) - g_\phi(z(t_i)) \right\|_2^2. \tag{88}$$

6.3 NODE-RNN for Irregular Sampling

A standard RNN update is

$$h_t = \mathrm{RNN}(h_{t-\Delta t}, x_t). \tag{89}$$

This update usually assumes a discrete sequence structure. NODE-RNN introduces continuous evolution between observations:

$$\widetilde{h}_t = \mathrm{ODESolve}(h_{t-\Delta t}, f_\theta, t - \Delta t, t), \tag{90}$$

$$h_t = \mathrm{RNN}(\widetilde{h}_t, x_t). \tag{91}$$

The ODE part evolves the hidden state through elapsed time, while the RNN part updates the hidden state when a new observation arrives. This is useful for irregularly sampled time series because the elapsed time between observations is explicitly modeled.

7 Implicit Neural Representations

7.1 Basic Definition

An Implicit Neural Representation models a signal as a continuous function of time:

$$f_\theta : \mathbb{R} \to \mathbb{R}^d, \qquad t \mapsto f_\theta(t). \tag{92}$$

The prediction is

$$\widehat{x}(t) = f_\theta(t). \tag{93}$$

Rather than storing a time series as a finite vector

$$[x_1, x_2, \ldots, x_T], \tag{94}$$

an INR stores the signal implicitly through the neural network parameters θ.

If we observe

$$\{(t_i, x_i)\}_{i=1}^N, \tag{95}$$

we train the INR by solving

$$\boxed{\min_\theta \sum_{i=1}^N \|f_\theta(t_i) - x_i\|_2^2 .} \tag{96}$$

After training, the model can be queried at any timestamp $t^\star$:

$$\widehat{x}(t^\star) = f_\theta(t^\star). \tag{97}$$

This makes INRs natural tools for interpolation, imputation, and continuous-time reconstruction.

8 Time Embedding Layers

Feeding raw time $t \in \mathbb{R}$ as a single scalar can be weak. Many signals contain periodic, high-frequency, or multi-scale components. Therefore, the first layer of an INR often embeds time into a richer representation.

A common Fourier feature embedding is

$$h_1(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \ldots, \sin(\omega_K t), \cos(\omega_K t)] . \tag{98}$$

The frequencies $\omega_1, \ldots, \omega_K$ may be fixed, randomly sampled, or learned.

Then the model becomes

$$\widehat{x}(t) = f_\theta(h_1(t)). \tag{99}$$

The purpose of this embedding is to provide a frequency-based representation of time. Low frequencies capture slow trends, while high frequencies capture rapid oscillations.

Using both sine and cosine allows phase shifts because

$$\sin(\omega t + \varphi) = \cos(\varphi)\sin(\omega t) + \sin(\varphi)\cos(\omega t). \tag{100}$$

9 SIRENs: Sinusoidal Representation Networks

9.1 Definition

A SIREN is an INR where the neural network uses sine activation functions. A standard MLP layer is

$$h^{(\ell+1)} = \sigma\left(W^{(\ell)}h^{(\ell)} + b^{(\ell)}\right), \tag{101}$$

where σ might be ReLU, tanh, or sigmoid. A SIREN layer is

$$\boxed{h^{(\ell+1)} = \sin\left(W^{(\ell)}h^{(\ell)} + b^{(\ell)}\right).} \tag{102}$$

For scalar time, one hidden neuron computes

$$h_j(t) = \sin(w_j t + b_j). \tag{103}$$

Here w_j behaves like a frequency and b_j behaves like a phase shift.

9.2 Fourier Basis Decomposition

A Fourier basis decomposition represents a signal as a sum of sine and cosine waves:

$$x(t) \approx a_0 + \sum_{k=1}^{K} \left[a_k \cos(\omega_k t) + b_k \sin(\omega_k t)\right]. \tag{104}$$

Thus a complex signal is decomposed into simple oscillations at different frequencies.

SIRENs are linked to this idea because each hidden neuron computes a learned sinusoidal component

$$\sin(w_j t + b_j). \tag{105}$$

Classical Fourier decomposition uses fixed basis functions. SIRENs use learned frequencies, phases, and nonlinear combinations. Therefore SIRENs can be interpreted as learnable Fourier-like representations.

9.3 Derivatives

Sine activations are smooth and have nonzero higher-order derivatives:

$$\frac{\mathrm{d}}{\mathrm{d}t}\sin(wt + b) = w\cos(wt + b), \tag{106}$$

$$\frac{\mathrm{d}^2}{\mathrm{d}t^2}\sin(wt + b) = -w^2\sin(wt + b). \tag{107}$$

This is useful when the learned signal must have meaningful derivatives, for example when modeling physical quantities or smooth continuous-time signals. ReLU networks, by contrast, have zero second derivatives almost everywhere.

9.4 SIREN Architecture

A typical SIREN is

$$h^{(0)} = t, \tag{108}$$

$$h^{(\ell+1)} = \sin\left(W^{(\ell)}h^{(\ell)} + b^{(\ell)}\right), \qquad \ell = 0, \dots, L-1, \tag{109}$$

$$\widehat{x}(t) = W^{(L)}h^{(L)} + b^{(L)}. \tag{110}$$

The training objective is

$$\min_{\theta} \sum_{i=1}^{N} \|f_\theta(t_i) - x_i\|_2^2 . \tag{111}$$

10 Modulated INRs

10.1 Why Modulation Is Needed

A basic INR represents one signal:

$$f_\theta : \mathbb{R} \to \mathbb{R}^d, \qquad t \mapsto f_\theta(t). \tag{112}$$

In practice, we often have a dataset of time series

$$x^{(1)}(t), x^{(2)}(t), \dots, x^{(N)}(t). \tag{113}$$

We therefore introduce a latent code $z_i \in \mathbb{R}^p$ for each series. The modulated INR is

$$f_\theta : \mathbb{R} \times \mathbb{R}^p \to \mathbb{R}^d, \qquad (t, z_i) \mapsto f_\theta(t, z_i). \tag{114}$$

Thus

$$\widehat{x}^{(i)}(t) = f_\theta(t, z_i). \tag{115}$$

The code z_i summarizes the content of the i-th time series: amplitude, trend, frequency, phase, shape, and other characteristics.

11 Feature-Wise Linear Modulation

11.1 FiLM Formula

Feature-Wise Linear Modulation, or FiLM, modulates hidden activations using a latent code. Suppose a hidden layer of the INR produces

$$h_\ell^{\mathrm{INR}}(t) \in \mathbb{R}^{m_\ell}. \tag{116}$$

A modulation network takes z_i as input and outputs

$$\gamma_\ell(z_i) \in \mathbb{R}^{m_\ell}, \qquad \beta_\ell(z_i) \in \mathbb{R}^{m_\ell}. \tag{117}$$

Then

$$\boxed{h_\ell^{\mathrm{mod}}(t, z_i) = \gamma_\ell(z_i) \odot h_\ell^{\mathrm{INR}}(t) + \beta_\ell(z_i).} \tag{118}$$

Here $\odot$ denotes componentwise multiplication. For each feature j,

$$h_{\ell,j}^{\mathrm{mod}}(t, z_i) = \gamma_{\ell,j}(z_i) h_{\ell,j}^{\mathrm{INR}}(t) + \beta_{\ell,j}(z_i). \tag{119}$$

Thus each hidden feature receives its own scale and shift.

11.2 FiLM Workflow

For one time series i:

1. the series is summarized by a code z_i;
2. the modulation network computes
$$M_\psi(z_i) = \{\gamma_\ell(z_i), \beta_\ell(z_i)\}_{\ell=1}^{L}; \tag{120}$$
3. the INR receives a timestamp t;
4. each hidden layer is modulated using $\gamma_\ell(z_i)$ and $\beta_\ell(z_i)$;
5. the final output is
$$\widehat{x}^{(i)}(t) = f_{\theta,\psi}(t, z_i). \tag{121}$$

11.3 FiLM Training

Suppose the dataset contains
$$\mathcal{D}_i = \{(t_{ij}, x_{ij})\}_{j=1}^{m_i}, \qquad i = 1, \ldots, N. \tag{122}$$
The prediction is
$$\widehat{x}_{ij} = f_{\theta,\psi}(t_{ij}, z_i). \tag{123}$$
A typical objective is
$$\boxed{\mathcal{L}(\theta, \psi, \{z_i\}) = \sum_{i=1}^{N}\sum_{j=1}^{m_i} \|f_{\theta,\psi}(t_{ij}, z_i) - x_{ij}\|_2^2 + \lambda \sum_{i=1}^{N} \|z_i\|_2^2.} \tag{124}$$
Training learns:

- θ: the shared INR parameters;
- ψ: the modulation-network parameters;
- z_i: the latent code for each training series, unless z_i is produced by an encoder.

The gradient updates are
$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \tag{125}$$
$$\psi \leftarrow \psi - \eta \nabla_\psi \mathcal{L}, \tag{126}$$
$$z_i \leftarrow z_i - \eta \nabla_{z_i} \mathcal{L}. \tag{127}$$

12 Hypernetworks for INRs

12.1 Basic Idea

A hypernetwork is a neural network that generates or modifies the weights of another neural network. In the INR setting:

- the **hyponetwork** is the INR $f_\theta(t)$;
- the **hypernetwork** takes a code z_i and outputs a parameter modification.

For a dataset of time series, one wants a different INR for each series, but without training a completely separate model from scratch. Let θ be shared base INR parameters. The hypernetwork outputs

$$\Delta\theta_i = \psi_\eta(z_i). \tag{128}$$

Then the personalized INR parameters are

$$\boxed{\theta_i = \theta + \psi_\eta(z_i).} \tag{129}$$

The prediction is

$$\boxed{\widehat{x}^{(i)}(t) = f_{\theta+\psi_\eta(z_i)}(t).} \tag{130}$$

12.2 From $\mathcal{D}_i$ to z_i to $\psi_\eta(z_i)$

For one time series

$$\mathcal{D}_i = \{(t_{ij}, x_{ij})\}_{j=1}^{m_i}, \tag{131}$$

we first obtain a latent code $z_i \in \mathbb{R}^p$.

There are two common ways.

12.2.1 Encoder-Based Code

An encoder maps the observed series to a code:

$$z_i = E_\xi(\mathcal{D}_i). \tag{132}$$

Then the hypernetwork maps this code to a parameter update:

$$\Delta\theta_i = \psi_\eta(z_i). \tag{133}$$

Hence

$$\mathcal{D}_i \longrightarrow E_\xi \longrightarrow z_i \longrightarrow \psi_\eta(z_i) \longrightarrow \theta_i = \theta + \psi_\eta(z_i). \tag{134}$$

12.2.2 Optimization-Based Code

Alternatively, each training series has a directly learnable code z_i. There is no encoder. The data $\mathcal{D}_i$ influences z_i through the reconstruction loss:

$$\sum_{j=1}^{m_i} \left\| f_{\theta+\psi_\eta(z_i)}(t_{ij}) - x_{ij} \right\|_2^2. \tag{135}$$

Gradient descent adjusts z_i so that the personalized INR fits the observed time series.

12.3 Hypernetwork Training

The training objective is

$$\boxed{\mathcal{L}(\theta, \eta, \{z_i\}) = \sum_{i=1}^{N} \sum_{j=1}^{m_i} \left\| f_{\theta+\psi_\eta(z_i)}(t_{ij}) - x_{ij} \right\|_2^2 + \lambda \sum_i \|z_i\|_2^2 + \rho \sum_i \|\psi_\eta(z_i)\|_2^2.} \tag{136}$$

The first term measures reconstruction error. The second term regularizes the latent codes. The third term prevents excessively large parameter modifications.

The full forward pass is

$$\boxed{\mathcal{D}_i \to z_i \to \psi_\eta(z_i) \to \theta_i = \theta + \psi_\eta(z_i) \to f_{\theta_i}(t) \to \widehat{x}_i(t).} \tag{137}$$

13 HyperTime and TimeFlow

13.1 HyperTime

HyperTime is an INR-based time-series model that uses an encoder to produce a latent representation. At a high level:

1. the observed time series is passed through a set encoder;
2. the encoder produces a code z;
3. the code is passed to a hypernetwork;
4. the hypernetwork generates the parameters of an INR, typically SIREN-style;
5. the predicted time series is compared with the ground truth.

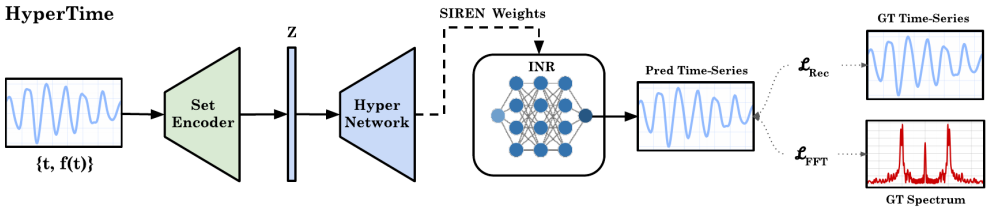
The model also uses an additional frequency-based loss so that the generated signal matches the spectral content of the true signal.

Typical TS forecasting INRs

Implicit Neural Representations

HyperTime

- Generates an encoding z per timestamp
- Requires global pooling along the time axis (or fixed number of observations per series)
- Uses an additional frequency-based loss



Source: "HyperTime: Implicit Neural Representation for Time Series", NeurIPS Workshop, 2022

Romain Tavenard Deep Learning for Time Series - Continuous-Time Models 19 / 20

Figure 1: HyperTime architecture. The observed time series is encoded into a latent code z, which is passed through a hypernetwork that generates SIREN/INR weights. The model uses both reconstruction loss and a frequency-domain loss.

13.2 TimeFlow

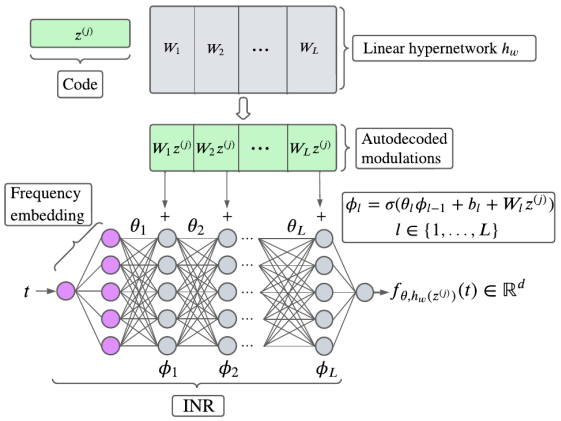
TimeFlow uses activation modulation, in the spirit of FiLM, rather than generating full INR weights. Its key difference is how the latent code is obtained. Instead of being output by an encoder, the code z is optimized through a few gradient-descent steps. This makes the method flexible for irregularly sampled time series because it does not require a fixed number of observations, a fixed input grid, or global pooling.

Typical TS forecasting INRs

Implicit Neural Representations

TimeFlow

- Activation modulations (*à la* FiLM)
- Code z is optimized through few-step gradient descent (not output by an encoder)
  $\rightarrow$ no constraint on the input time grid, no need for pooling



Source: "Time Series Continuous Modeling for Imputation and Forecasting with Implicit Neural Representations", TMLR'24

Romain Tavenard Deep Learning for Time Series - Continuous-Time Models 20 / 20

Figure 2: TimeFlow architecture. The latent code z is optimized through a few gradient-descent steps and then used to modulate the INR activations in a FiLM-like way. This avoids fixed-grid constraints and removes the need for global pooling.

14 Summary

The chapter can be summarized as follows:

- ODEs model continuous-time dynamics through

$$\dot{x}(t) = f(x(t), t). \tag{138}$$

- ODE solvers approximate trajectories at discrete points. Euler is simple but low-order; Runge–Kutta methods are more accurate but use more function evaluations.
- Neural ODEs replace the unknown vector field by a neural network:

$$\dot{z}(t) = f_\theta(z(t), t). \tag{139}$$

- Latent NODEs encode observed histories into latent states, evolve them continuously, then decode them back to data space.
- NODE-RNNs handle irregular sampling by evolving hidden states continuously between observations and updating them when data arrive.
- INRs represent time series as continuous neural functions:

$$\widehat{x}(t) = f_\theta(t). \tag{140}$$

- Fourier embeddings and SIRENs improve the ability of INRs to represent oscillatory and high-frequency signals.
- Modulated INRs use a latent code z_i to represent a family of time series.
- FiLM modulates activations:

$$h^{\text{mod}} = \gamma(z) \odot h + \beta(z). \tag{141}$$

- Hypernetworks modulate parameters:

$$\theta_i = \theta + \psi(z_i). \tag{142}$$

The main idea to remember is:

Continuous-time models replace fixed discrete sequences by neural functions or dynamics that can be evaluat

(143)